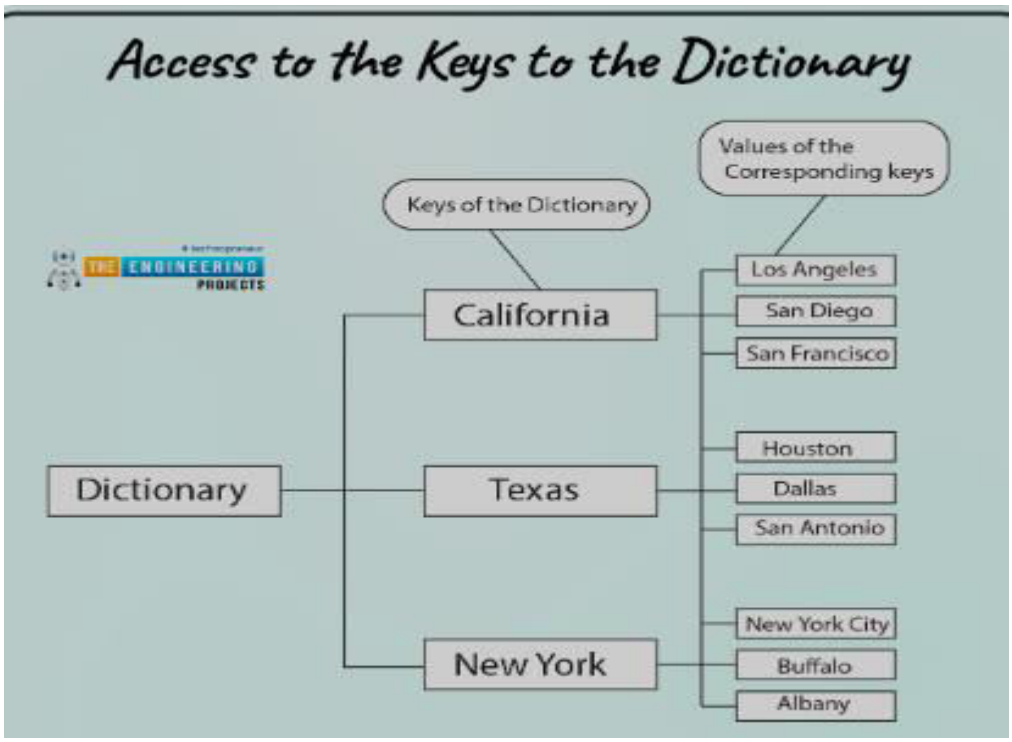
Dictionary in python

Dictionary:

A dictionary in Python is an **unordered collection of key-value pairs**. It is defined using curly braces {} and colons : to separate keys and values.



Example:

```python
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
print(my_dict)  # Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Dictionary Comprehension:

Dictionary comprehension is a concise way to create dictionaries using an iterable. It follows the syntax:

```python
{key_expression: value_expression for item in iterable}
```

Example:

```python
# Create a dictionary with squares of numbers
squares = {x: x**2 for x in range(1, 6)}
print(squares)  # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Create a Dictionary Using Variables:

You can create a dictionary using variables for keys and values.

Example:

```python
name = "Alice"
age = 25
city = "New York"

my_dict = {"name": name, "age": age, "city": city}
print(my_dict)  # Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Keys:Values Concept

- **Keys**: Unique identifiers for values in a dictionary. Keys must be immutable (e.g., strings, numbers, or tuples).
- **Values**: Data associated with keys. Values can be of any data type (e.g., strings, numbers, lists, or even other dictionaries).

Example:

```python
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
print(my_dict["name"])  # Output: Alice (accessing value using key)
```

Dictionary Methods

len()

Returns the number of key-value pairs in the dictionary.

Example:

```python
my_dict = {"name": "Alice", "age": 25}
print(len(my_dict))  # Output: 2
```

keys()

Returns a view of all keys in the dictionary.

Example:

```python
print(my_dict.keys())  # Output: dict_keys(['name', 'age'])
```

values()

Returns a view of all values in the dictionary.

Example:

```python
print(my_dict.values())  # Output: dict_values(['Alice', 25])
```

items()

Returns a view of all key-value pairs as tuples.

Example:

```python
print(my_dict.items())  # Output: dict_items([('name', 'Alice'), ('age', 25)])
```

get()

Retrieves the value for a key. If the key does not exist, it returns None or a default value.

Example:

```python
print(my_dict.get("name"))  # Output: Alice
print(my_dict.get("address", "Not Found"))  # Output: Not Found
```

pop()

Removes a key-value pair from the dictionary and returns the value.

Example:

```python
age = my_dict.pop("age")
print(age)  # Output: 25
print(my_dict)  # Output: {'name': 'Alice'}
```

update()

Updates the dictionary with key-value pairs from another dictionary or iterable.

Example:

```python
my_dict.update({"city": "New York", "age": 30})
print(my_dict)  # Output: {'name': 'Alice', 'city': 'New York', 'age': 30}
```

Comparison of Data Structures

Feature	List	Tuple	Dictionary	Set

Feature	List	Tuple	Dictionary	Set
Mutable	Yes	No	Yes (keys: No)	Yes
Ordered	Yes	Yes	No (Python 3.7+: Yes)	No
Duplicates Allowed	Yes	Yes	Keys: No, Values: Yes	No
Use Case	Ordered collection	Immutable collection	Key-value pairs	Unique elements

Introduction to range()

The range() function generates a sequence of numbers. It is commonly used in loops.

Syntax

```python
range(start, stop, step)
```

- start: Starting number (inclusive, default is 0).
- stop: Ending number (exclusive).
- step: Difference between numbers (default is 1).

Example:

```python
for i in range(5):
    print(i)  # Output: 0 1 2 3 4
```

Pass range() in a List

You can convert a range() object to a list using the list() function.

Example:

```python
numbers = list(range(1, 6))
print(numbers)  # Output: [1, 2, 3, 4, 5]
```

range () Arguments

- **Single Argument**: range(stop)
  Generates numbers from 0 to stop-1.

```python
print(list(range(5)))  # Output: [0, 1, 2, 3, 4]
```

- **Two Arguments**: range(start, stop)
  Generates numbers from start to stop-1.

```python
print(list(range(2, 6)))  # Output: [2, 3, 4, 5]
```

- **Three Arguments**: range(start, stop, step)
  Generates numbers from start to stop-1 with a step size of step.

```python
print(list(range(1, 10, 2)))  # Output: [1, 3, 5, 7, 9]
```

For Loop Introduction Using range()

A for loop iterates over a sequence (e.g., a list, tuple, or range() object).

Example:

```python
# Print numbers from 0 to 4
for i in range(5):
    print(i)  # Output: 0 1 2 3 4
```

You can also use range() to iterate over a list by index:

```python
fruits = ["apple", "banana", "cherry"]
for i in range(len(fruits)):
    print(fruits[i])  # Output: apple banana cherry
```

DICTIONARY INTERVIEW QUESTIONS AND ANSWERS

BASIC LEVEL QUESTIONS

Q1. What is a dictionary in Python?

Ans: A dictionary is an **unordered collection of key-value pairs**. It is defined using curly braces {} and colons : to separate keys and values. Keys must be unique and immutable (e.g., strings, numbers, or tuples), while values can be of any data type.

Q2. How do you access a value in a dictionary?

Ans: You can access a value in a dictionary using its key. If the key does not exist, it raises a KeyError. To avoid this, use the get() method.

Q3. How do you add or update a key-value pair in a dictionary?

Ans: You can add or update a key-value pair by assigning a value to a key. If the key already exists, its value is updated. If the key does not exist, it is added to the dictionary.

Q4. How do you remove a key-value pair from a dictionary?

Ans: You can remove a key-value pair using:

- pop(key): Removes the key and returns its value.
- del: Deletes the key-value pair.

Q5. How do you check if a key exists in a dictionary?

Ans: You can check if a key exists in a dictionary using the in keyword.

Example:

```
my_dict = {"name": "Alice", "age": 25}
if "name" in my_dict:
    print("Key exists!")  # Output: Key exists!
else:
    print("Key does not exist.")
```

INTERMEDIATE LEVEL QUESTIONS

Q6. How can you merge two dictionaries in Python?

Ans: You can merge two dictionaries using:

1. The update() method (modifies the original dictionary).
2. The ** unpacking operator (creates a new dictionary).

Q7. How can you use dictionary comprehension to create a dictionary?

Ans: Dictionary comprehension is a concise way to create dictionaries using an iterable. It follows the syntax:

{key_expression: value_expression for item in iterable}

Example:

```
# Create a dictionary with squares of numbers
```

```
squares = {x: x**2 for x in range(1, 6)}
print(squares)  # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}


# Create a dictionary from two lists
keys = ["a", "b", "c"]
values = [1, 2, 3]
my_dict = {k: v for k, v in zip(keys, values)}
print(my_dict)  # Output: {'a': 1, 'b': 2, 'c': 3}
```

Q8. How do you handle missing keys in a dictionary?

Ans: You can handle missing keys using:

1. The get() method (returns None or a default value if the key is missing).
2. The defaultdict from the collections module (provides a default value for missing keys).

Q9. How do you sort a dictionary by keys or values?

Ans: You can sort a dictionary by keys or values using the sorted() function. The result is a list of tuples, which can be converted back to a dictionary.

Example:

```
my_dict = {"b": 2, "a": 1, "c": 3}
# Sort by keys
sorted_by_keys = {k: my_dict[k] for k in sorted(my_dict)}
print(sorted_by_keys)  # Output: {'a': 1, 'b': 2, 'c': 3}
# Sort by values
sorted_by_values = {k: v for k, v in sorted(my_dict.items(), key=lambda item: item[1])}
print(sorted_by_values)  # Output: {'a': 1, 'b': 2, 'c': 3}
```

Q10. How do you use dictionaries to count the frequency of elements in a list?

Ans: You can use a dictionary to count the frequency of elements in a list by iterating through the list and updating the dictionary.

Example:

```
# Using a loop
my_list = ["apple", "banana", "apple", "orange", "banana", "apple"]
frequency = {}
for item in my_list:
```

```
    if item in frequency:
        frequency[item] += 1
    else:
        frequency[item] = 1
print(frequency)  # Output: {'apple': 3, 'banana': 2, 'orange': 1}
# Using collections.Counter
from collections import Counter
frequency = Counter(my_list)
print(frequency)  # Output: Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

ADVANCE LEVEL QUESTIONS

Q11. How can you use dictionaries to implement a switch-case-like behaviour in Python?

Ans: Python does not have a built-in switch-case statement, but you can simulate it using a dictionary. This approach is efficient and clean.

Example:

```
def switch_case(argument):
    # Define a dictionary to map cases to functions
    switcher = {
        1: "Case 1 executed",
        2: "Case 2 executed",
        3: "Case 3 executed",
    }
    # Get the function from the dictionary, or a default case
    return switcher.get(argument, "Invalid case")

print(switch_case(2))  # Output: Case 2 executed
print(switch_case(4))  # Output: Invalid case
```

Q12. How can you use dictionaries to memorize functions for optimization?

Ans: Memorization is a technique to store the results of expensive function calls and reuse them when the same inputs occur again. Dictionaries are ideal for this because of their fast lookups.

Example:

```
# Memorization using a dictionary
```

```
def fibonacci(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 2:
        return 1
    memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo)
    return memo[n]
print(fibonacci(10))  # Output: 55
print(fibonacci(50))  # Output: 12586269025 (computed efficiently)
```

Q13. How can you use dictionaries to implement a priority queue?

Ans: While Python's heapq module is typically used for priority queues, you can simulate one using dictionaries combined with sorting.

Example:

```
# Simulating a priority queue using a dictionary
priority_queue = {
    "task1": 3,
    "task2": 1,
    "task3": 2,
}
# Get tasks in priority order
sorted_tasks = sorted(priority_queue.items(), key=lambda x: x[1])
for task, priority in sorted_tasks:
    print(f"Executing {task} with priority {priority}")
# Output:
# Executing task2 with priority 1
# Executing task3 with priority 2
# Executing task1 with priority 3
```

Q14. How can you use dictionaries to implement a graph data structure?

Ans: A graph can be represented as a dictionary where keys are nodes, and values are lists of adjacent nodes (for adjacency lists).

Example:

```
# Representing a graph using a dictionary
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"],
}
# Function to traverse the graph (BFS)
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node)
            visited.add(node)
            queue.extend(graph[node])
bfs(graph, "A")
# Output: A B C D E F
```

Q15. How can you use dictionaries to implement a sparse matrix?

Ans: A sparse matrix (a matrix with mostly zero values) can be efficiently represented using a dictionary, where keys are tuples of indices (row, col) and values are non-zero elements.

Example:

```
# Representing a sparse matrix using a dictionary
sparse_matrix = {
    (0, 0): 1,
    (1, 1): 2,
    (2, 2): 3,
```

```
}
# Accessing elements
def get_matrix_value(matrix, row, col):
    return matrix.get((row, col), 0)
print(get_matrix_value(sparse_matrix, 0, 0))  # Output: 1
print(get_matrix_value(sparse_matrix, 1, 2))  # Output: 0 (default value)
```